Design Document

PS15 — Route Optimisation with a Genetic Algorithm
ACI Assignment 1 | Team 111

---

1. Problem Statement

Given a set of N cities with (x, y) coordinates, find a closed tour (Travelling Salesman variant) that visits every city exactly once and returns to the origin, subject to two hard constraints:

- **Time constraint:** total travel time ≤ T_{max}
- **Cost constraint:** total route distance ≤ C_{max}

The optimisation objective is to minimise total distance (and hence travel time, since time = distance / speed). The solution must use a Genetic Algorithm with permutation encoding, ordered crossover, swap mutation, and rank selection, and must be compared against random search and hill climbing baselines.

2. Input / Output Format

2.1 Input file (inputPS15.txt)

Plain text, one entry per line. Whitespace and '=' are flexible:

```
Tmax = 150
Cmax = 1100
City Coordinates:
0  950.5  2500.4  Delhi
1  803.8  2312.3  Jaipur
...
```

City IDs must be contiguous integers starting at 0. If no file is supplied the program generates a random 20-city instance.

2.2 Output file (outputPS15.txt)

The output file records: best route with city names, total distance, total time, constraint satisfaction status, GA parameters, a performance comparison table (Random Search / Hill Climbing / GA), and a convergence summary. Three PNG plots are also saved (best_route.png, distance_convergence.png, time_convergence.png).

3. Data Structures

3.1 Route (permutation list)

A route is a plain Python list of N city IDs, e.g. [3, 7, 1, 0, …], representing the order in which cities are visited. The tour is implicitly closed — the last city connects back to the first. Permutation encoding guarantees every city appears exactly once.

3.2 Population (bounded container)

The class **Population** wraps a fixed-capacity list of routes. It raises OverflowError on add() when full and IndexError on remove() when empty, satisfying the brief's requirement for capacity error reporting. Capacity =

POP_SIZE (default 120).

3.3 Distance matrix

An N×N NumPy array pre-computed once before the GA runs. Entry dmat[i][j] is the Euclidean distance between cities i and j. Pre-computation converts $O(N^2)$ repeated sqrt calls into O(1) lookups during fitness evaluation.

3.4 Evaluation result (dict)

Each route evaluation returns a dictionary with keys: route, distance, time, combined, combined_clean, fitness, feasible. Passing a dict avoids re-computing the same route multiple times per generation.

4. Algorithm Design

4.1 Encoding (permutation)

Each individual is a list of N city IDs in visit order. init_population() creates POP_SIZE random shuffles of [0 … N-1]. Permutation encoding makes it impossible to visit a city twice, so no repair step is needed.

4.2 Fitness function

fitness = 1 / (W_DIST × dist + W_TIME × time + penalty)

Higher fitness is better (minimisation becomes maximisation). A penalty term — PENALTY_BIG × amount_of_violation — is added for breaching T_{max} or C_{max}. The penalty is proportional (not binary), so infeasible routes degrade gracefully and can still propagate useful genetic material.

4.3 Rank selection

Parents are chosen by rank, not raw fitness. Individuals are sorted worst-to-best and assigned ranks 1 … N. Selection probability is proportional to rank. Rank selection prevents premature convergence caused by a single very-fit individual dominating early generations.

4.4 Ordered Crossover (OX)

Two random cut points a, b are chosen. The child inherits the slice [a:b+1] from parent 1 unchanged. The remaining positions are filled left-to-right with cities from parent 2, skipping any city already present in the slice. The result is always a valid permutation.

4.5 Swap mutation

With probability MUTATION_RT (default 0.20) two random positions in the child are swapped. The tour remains a valid permutation after mutation. The rate is kept at 20 % — high enough to maintain diversity, low enough not to turn the GA into random search.

4.6 Elitism

The ELITES (default 2) best routes from generation g are copied unchanged into generation g+1 before breeding begins. This ensures the best solution found so far is never lost to crossover or mutation.

5. Tunable Parameters

Parameter	Value	Purpose
SPEED	60.0 km/h	Converts distance to travel time
W_DIST	1.0	Weight on distance in combined objective
W_TIME	1.0	Weight on time in combined objective
PENALTY_BIG	1000.0	Proportional penalty for constraint violations
POP_SIZE	120	Number of routes in the population
GENERATIONS	500	Number of GA generations
MUTATION_RT	0.20	Probability of swap mutation per child
ELITES	2	Routes copied unchanged each generation
RANDOM_ITER	5000	Iterations for random search baseline
HILL_ITER	5000	Iterations for hill climbing baseline

6. Baseline Algorithms

6.1 Random Search

Generates RANDOM_ITER random permutations and keeps the best. It has no memory between iterations. It serves as a lower-bound reference: any intelligent algorithm should beat it decisively.

6.2 Hill Climbing

Starts from one random tour and applies forced swap mutations. Accepts a new tour only if it strictly improves fitness. Converges quickly to a local optimum and cannot escape it. Mirrors the instructor's template structure for direct comparability.

7. Alternate Modelling Note

In the current model, time is derived directly from distance (time = distance / SPEED), so T_{max} and C_{max} constrain the same quantity scaled differently. The optimisation landscape has a single trade-off axis.

A richer model would assign each road segment its own speed limit and toll, independent of its Euclidean length. Under that model the shortest tour is not necessarily the fastest or cheapest, creating a genuine bi-objective trade-off. The GA encoding and operators would remain the same; only the fitness function and input format would change. A Pareto-front approach (NSGA-II) would then be appropriate.

8. Execution Flow

1. Read inputPS15.txt (or generate a random instance).
2. Build the N×N distance matrix once.
3. Run the Genetic Algorithm for GENERATIONS iterations.
4. Run Random Search for RANDOM_ITER iterations.
5. Run Hill Climbing for HILL_ITER iterations.
6. Compute normalised fitness scores for all three methods.
7. Print a comparison table to the console.
8. Save three PNG plots: best route map, distance convergence, time convergence.
9. Write outputPS15.txt with full results and constraint status.

9. Key Design Decisions

Why proportional penalty instead of death penalty?

Completely discarding infeasible routes early in the run risks losing diversity. A proportional penalty keeps them in the gene pool at a fitness disadvantage, so the GA can still use their structural information while favouring feasible routes.

Why rank selection instead of roulette-wheel (fitness-proportional)?

Raw fitness values can vary by orders of magnitude across generations, causing one elite individual to dominate selection. Rank selection compresses the selection pressure to a linear scale and maintains population diversity throughout the run.

Why elitism = 2?

A single elite is sufficient to preserve the best solution; two elites add a small buffer without homogenising the population. Larger elite sizes would slow exploration.
